

Impact of DDR4 memory speed and latency on gaming performance



2020/7/19

Initial experiment III

Introduction

In the first experiment we looked at the relationship between Thermal Design Power and power consumption, and we now know laptop computers usually stay within its TDP while desktop processors can sometimes shoot above its TDP.

In the second experiment we looked at the performance and efficiency difference between the two systems.

In this last experiment we will look deeper down into our desktop system and focus on different memory speed and latency setup for gaming performance. Memory latency and memory speed, which has the bigger impact?

Experiment Design

DDR4 memories runs at certain clock speed and CL values (latency). The same DDR4 memory can usually opt for higher clock speed or lower CL values. Higher memory speed provides higher memory bandwidth, while lower CL values provides lower latency with the same memory bandwidth.

Setup A: DDR4 2666 CL16

Setup A is the default setting on my memories. Set as standard.

Setup B: DDR4 2666 CL12

Setup B is the lowest achievable latency while maintaining the same memory speed as setup A.

Setup C: DDR4 3466 CL16

Setup C is the highest achievable memory speed while maintaining the same latency as setup A.

Overall	Desktop
Name	DIY Desktop
Processor	Core i7-9800X
TDP	165 Watts
Core/Threads	8C / 16T
Clock	3.8-4.5GHz
uarch	Skylake-X
L3\$	16.5MiB
Motherboard	ASUS Prime X299
Memory	4-Channel DDR4
GPU	Radeon RX 590

This system is tested (reset) as factory configured default to make sure they follow vendor's official specifica-

tions.

Tested program this time is the most popular first-person shooting game “Counter Strike Global Offensive”, which has quite low system requirement thus we can push for high frame rates with no bottlenecks on GPU side.

The test is conducted at 720p low settings to further remove GPU bottleneck, making the game complete CPU-bound.

Data Collection

Our test is done in all vs one fashion, with a balanced design.

Before we conduct the test, we need to decide how many runs we want to produce.

Because we have 3 settings so $a = 3$ levels. $K = a - 1 = 2$

Let's assume we want a 95% confidence level. So $\alpha = 0.05$. And the power of the test is desired to be 0.8, so $\beta = 0.2$. Because CPU frequency is not adjusted, the performance impact won't be huge. And CSGO has a very high frame rate (usually $>100\text{FPS}$). So we estimate $d=10$ (10FPS worthwhile detected), $\sigma=15$.

Required sample size should be:

$n = 18$ (17.64)

So we will have to perform the test at least 18 times for each setup

The test result is gathered through Radeon Software's integrated average frame rate monitor.

The game benchmark is run 18 times each with scripted savedata and map.

Analysis

After collecting all the data, the chart is shown as below (randomized)

```
csgo <- read.csv("csgo.csv")
set.seed(1)
dplyr::sample_n(csgo, 18)
```

##		X	FPS	Setup
## 1	4	250	A	
## 2	39	301	C	
## 3	1	236	A	
## 4	34	291	B	
## 5	23	305	B	
## 6	43	294	C	
## 7	14	269	A	
## 8	18	272	A	
## 9	33	280	B	
## 10	21	289	B	
## 11	45	308	C	
## 12	42	287	C	
## 13	10	255	A	
## 14	7	269	A	
## 15	9	242	A	
## 16	15	248	A	
## 17	44	306	C	
## 18	37	319	C	

```
csgo$Setup <- ordered(csgo$Setup,
                      levels = c("A", "B", "C"))
```

```
library(dplyr)
```

```
## Warning: package 'dplyr' was built under R version 4.0.2
```

```
##
```

```
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
## filter, lag
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
## intersect, setdiff, setequal, union
```

```
group_by(csgo, Setup) %>%
  summarise(
    count = n(),
    mean = mean(FPS, na.rm = TRUE),
    sd = sd(FPS, na.rm = TRUE)
  )
```

```
## `summarise()` ungrouping output (override with `.groups` argument)
```

```
## # A tibble: 3 x 4
```

```
##   Setup count mean    sd
```

```
##   <ord> <int> <dbl> <dbl>
```

```
## 1 A      18  258  12.6
```

```
## 2 B      18  286.  20.3
```

```
## 3 C      18  307.  10.2
```

Visualized data:

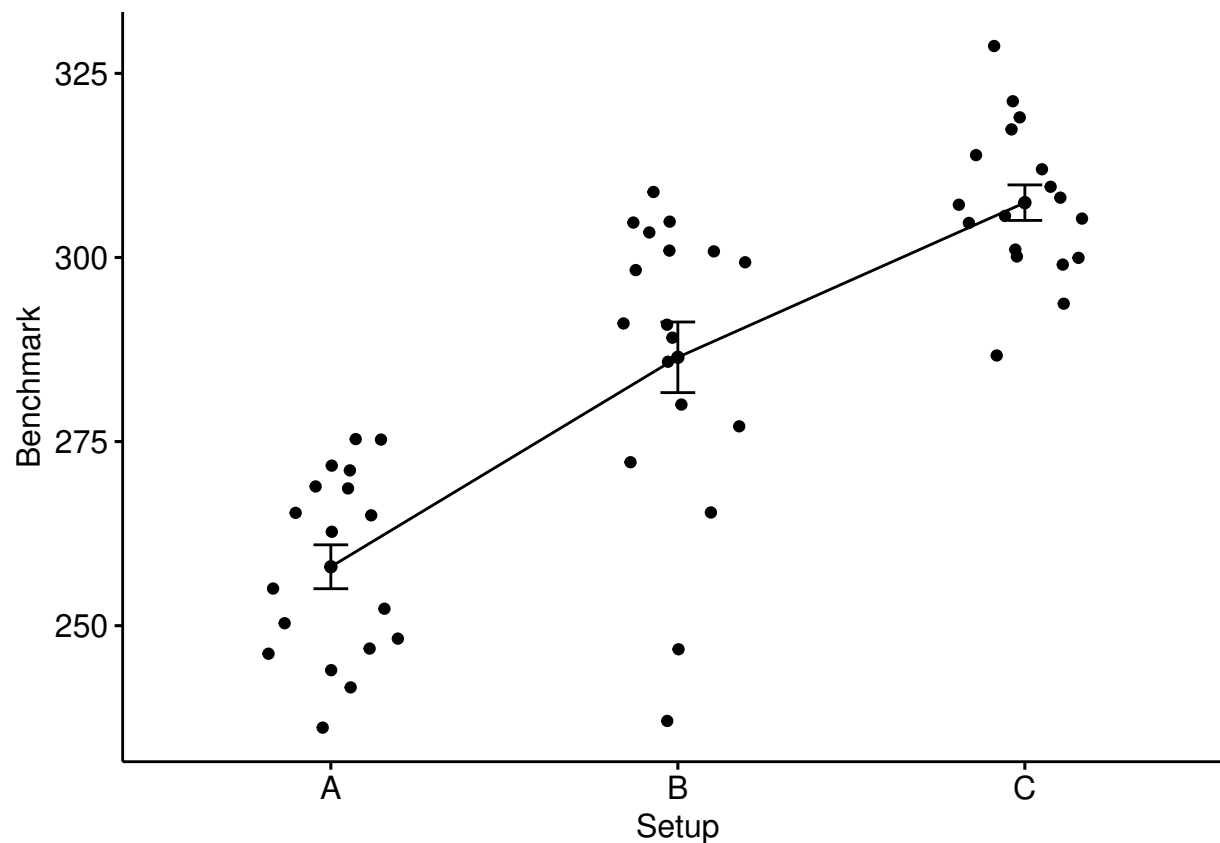
```
library("ggpubr")
```

```
## Warning: package 'ggpubr' was built under R version 4.0.2
```

```
## Loading required package: ggplot2
```

```
## Warning: package 'ggplot2' was built under R version 4.0.2
```

```
ggline(csgo, x = "Setup", y = "FPS",
       add = c("mean_se", "jitter"),
       order = c("A", "B", "C"),
       ylab = "Benchmark", xlab = "Setup")
```



Simultaneous Confidence Intervals Multiple Comparisons of Means: Dunnett Contrasts 95% family-wise confidence level Linear Hypotheses:

```
res.aov <- aov(FPS ~ Setup, data = csgo)
summary(res.aov)
```

```
##           Df Sum Sq Mean Sq F value    Pr(>F)
## Setup      2  22169   11085    49.02 1.33e-12 ***
## Residuals  51  11533     226
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
TukeyHSD(res.aov)
```

```
##   Tukey multiple comparisons of means
##     95% family-wise confidence level
##
## Fit: aov(formula = FPS ~ Setup, data = csgo)
##
## $Setup
##      diff      lwr      upr    p adj
## B-A 28.44444 16.344140 40.54475 0.0000020
## C-A 49.44444 37.344140 61.54475 0.0000000
## C-B 21.00000  8.899696 33.10030 0.0003223
```

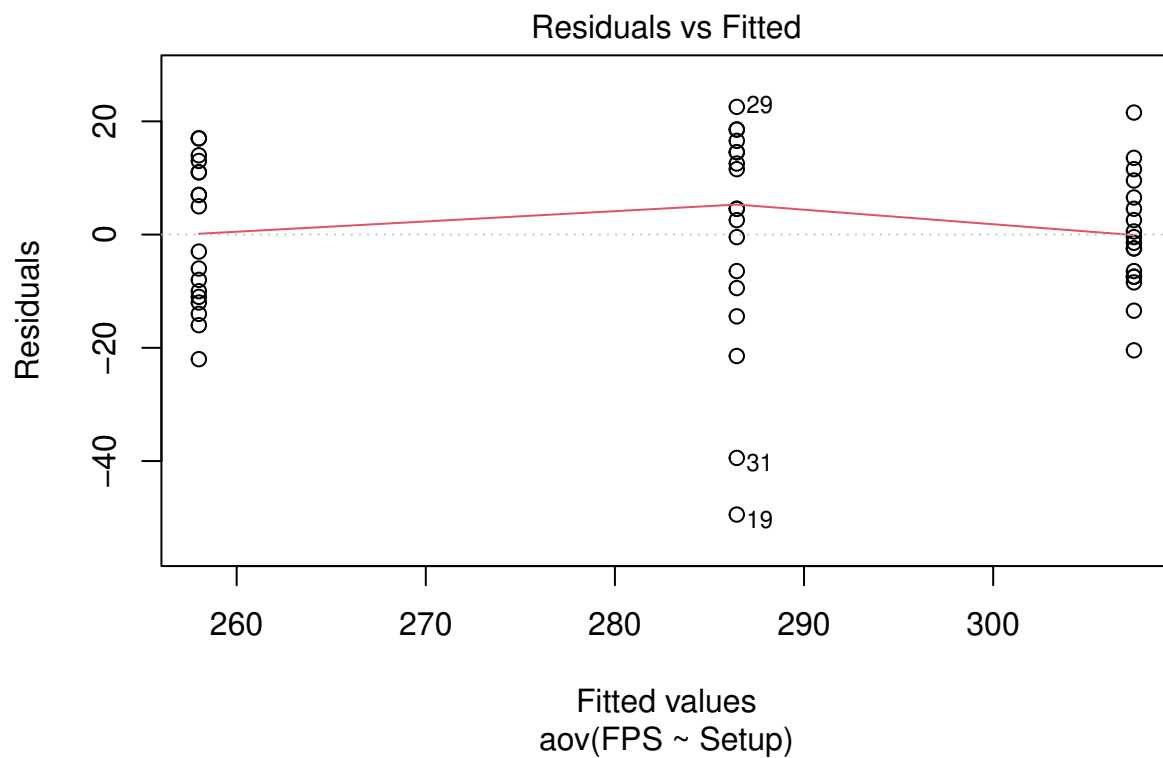
```
library(multcomp)
```

```
## Warning: package 'multcomp' was built under R version 4.0.2
```

```
## Loading required package: mvtnorm
## Loading required package: survival
## Loading required package: TH.data
## Warning: package 'TH.data' was built under R version 4.0.2
## Loading required package: MASS
##
## Attaching package: 'MASS'
## The following object is masked from 'package:dplyr':
##
##   select
##
## Attaching package: 'TH.data'
## The following object is masked from 'package:MASS':
##
##   geyser
```

Residual vs Fitted

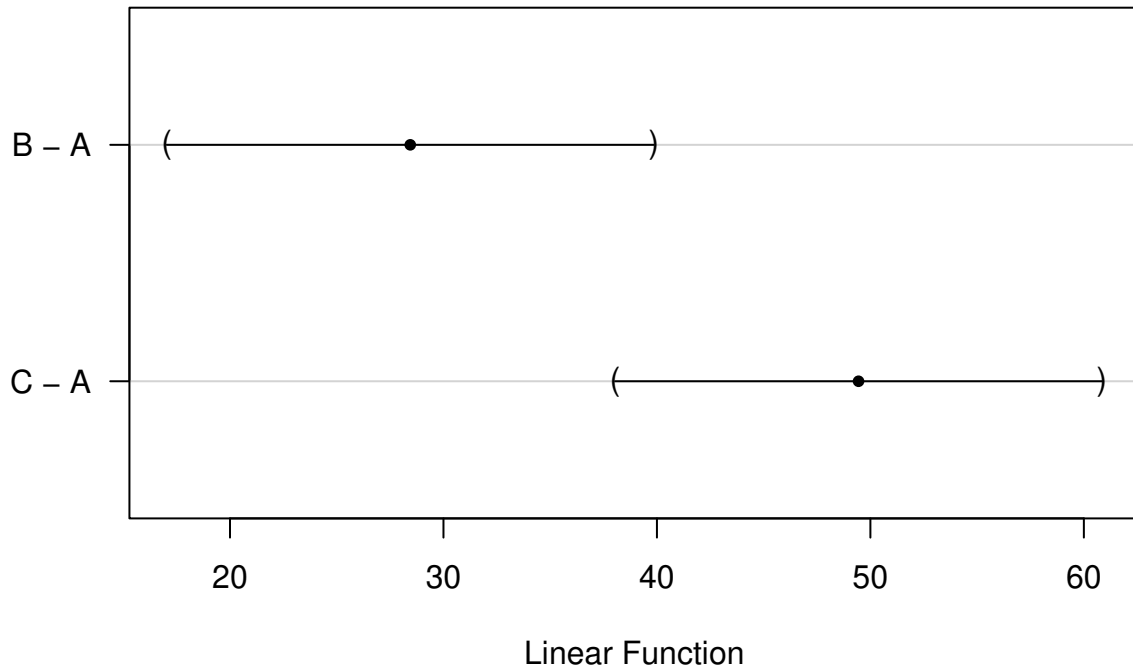
```
plot(res.aov, 1)
```



```
model2 <- aov(FPS ~ Setup, data = csgo)
mc2 <- glht(model2, linfct = mcp(Setup = "Dunnett"))
```

```
mc2_CI <- confint(mc2, level = 0.95)
plot(mc2_CI)
```

95% family-wise confidence level



The mean of setup A is 258FPS, and 286.44FPS for Setup B, and 307.44 for setup C. Standard deviation sigma for setup A is 12.64, and 20.34 for setup B and 10.23 for setup C.

Overall, Setup C has higher performance uplift from Setup A than Setup B, so memory speed does have a bigger impact than memory latency in this game. And the difference is quite big, around 20 FPS of difference.